Micro processors Systems

Flag Control, Jumps, Stack Operations, and x86 Assembly Interrupts

Lecturer: Dr. Hesham El Zouka

Flag Control Instructions

SF	ZF		AF		PF		CF

- LAHF Load AH from flags (AH) ← (Flags)
- SAHF Store AH into flags (Flags) ← (AH)

Flags affected: SF, ZF, AF, PF, CF

- CLC Clear Carry Flag (CF) ← 0
- STC Set Carry Flag (CF) ← 1
- CLI Clear Interrupt Flag (IF) ← 0
- STI Set interrupt flag (IF) ← 1

Example

```
LAHF
MOV [SI], AH
MOV AH, [DI]
SAHF
```

Compare Instruction

Mnemonic	Meaning	Format	Operation	Flags Affected
CMP	Compare	CMP D,S	(D) – (S) is used in setting or resetting the flags	CF, AF, OF, PF, SF, ZF

Destination	Source
Register	**Register**
Register	**Memory**
Memory	**Register**
Register	**Immediate**
Memory	**Immediate**
Accumulator	**Immediate**

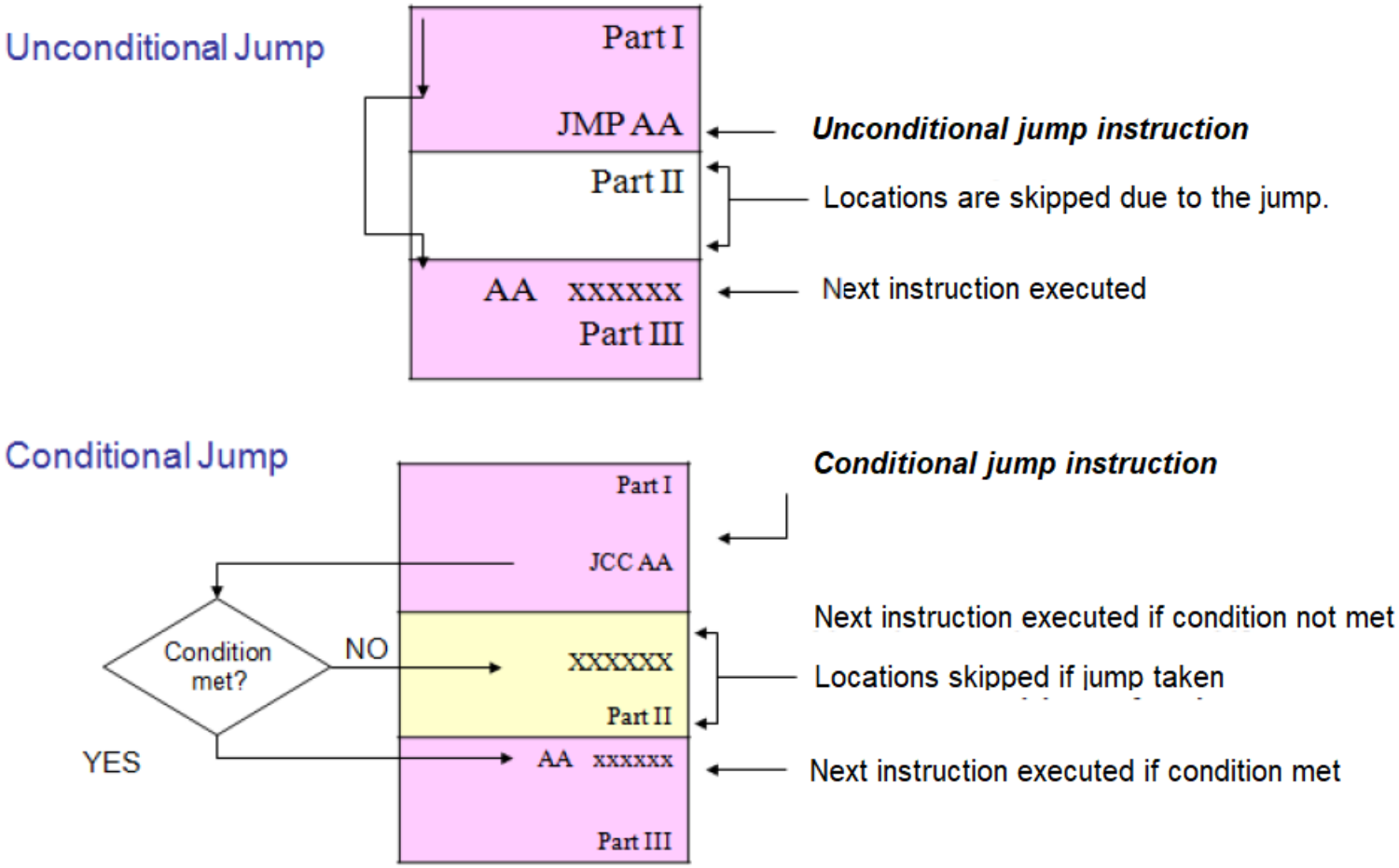
MOV AL, 7
CMP AL, 5

MOV AL, 99H; -103
CMP AL, 1BH; +27

Example:

```
DATA1        DW      235Fh
…
MOV AX, CCCCH
CMP AX, DATA1
JNC OVER
SUB AX, AX
OVER: INC DATA1
```

Jump Instructions



Unconditional Jump

Mnemonic	Meaning	Format	Operation	Flags Affected
JMP	Unconditional jump	JMP Operand	Jump is initiated to the address specified by the operand	None

Operand
Short-label
Near label
Far-label
Memptr16
Regptr16
Memptr32

JMP 1234H
JMP BX
JMP [BX]
JMP DWORD PTR [DI]

Example:

```asm
org 100h            ; Code origin for .COM programs

start:
    mov ah, 09h     ; DOS interrupt for displaying a string
    lea dx, msg     ; Load the address of the message
    int 21h         ; Call DOS to display the string

    mov ah, 16h     ; BIOS interrupt for keyboard input
    int 16h         ; Wait for a keypress and store it in AX

    mov ah, 09h     ; DOS interrupt to display a string
    lea dx, result  ; Load the address of the result message
    int 21h         ; Display the result message

    mov ah, 4Ch     ; DOS interrupt to terminate the program
    int 21h         ; End the program

msg db 'Press any key to continue...', '$'     ; Message to display
result db 0Dh, 0Ah, 'Key captured. Program ending.$' ; Result message
```

Conditional Jump

These flags are determined based on general comparison results.

Mnemonic	Description	Flag/Registers
JZ	Jump if ZERO	ZF=1
JE	Jump if EQUAL	ZF=1
JNZ	Jump if NOT ZERO	ZF=0
JNE	Jump if NOT EQUAL	ZF=0
JC	Jump if CARRY	CF=1
JNC	Jump if NO CARRY	CF=0
JCXZ	Jump if CX=0	CX=0
JECXZ	Jump if ECX=0	ECX=0
JP	Jump if PARITY EVEN	PF=1
JNP	Jump if PARITY ODD	PF=0

Jump Based on Unsigned Comparison

These flags are determined based on unsigned comparison operations.

Mnemonic	Description	Flag/Registers
JA	Jump if above op 1>op2	CF=0 and ZF=0
JNBE	Jump if not below or equal op1 not <= op2	CF=0 and ZF=0
JAE	Jump if above or equal op1>=op2	CF=0
JNB	Jump if not below op1 not <op2	CF=0
JB	Jump if below op1<op2	CF=1
JNAE	Jump if not above nor equal op1<op2	CF=1
JBE	Jump if below or equal Op1<=op2	CX=1 or ZF=1
JNA	Jump if not above Op1<=op2	CF=1 or ZF=1

Conditional Jump (Cont.)

Mnemonic	Description	Flag/Registers
JG	Jump if GREATER (op1 > op2)	SF=OF and ZF=0
JNLE	Jump if NOT LESS THAN or equal (op1 > op2)	SF=OF and ZF=0
JGE	Jump if GREATER THAN or equal (op1>= op2)	SF=OF
JNL	Jump if not LESS THA (op1 >= op2)	SF=OF
JL	Jump if LESS THAN (op1 < op2)	SF ≠ OF
JNGE	Jump if not GREATER THAN nor equal (op1<op2)	SF ≠ OF
JLE	Jump if LESS THAN or equal Op1<=op2	ZF=1 or SF ≠ OF
JNG	Jump if not GREATER THAN op1<=op2 Op1<=op2	ZF=1 or SF ≠ OF

JS	Jump if SIGNED (OP1 NEGATIVE)	SF = 1
JNS	Jump if NOT SIGNED (OP1 POSITIVE)	SF = 0
JO	Jump if OVERFLOW	OF = 1
JNO	Jump if NO OVERFLOW	OF = 0

Short Jumps

- Conditional Jump is a two byte instruction.
- In a jump backward the second byte is the 2's complement of the displacement value.
- To calculate the target the second byte is added to the IP of the instruction after the jump.
- Ex:

```
000D ADD  AL,[BX]
000F INC BX
0010 DEC CX
0011 JNZ 000D
0013
```

Short Jump 0013+FA (-6) = 0D

Example:

Write a program that adds 5 bytes of data and saves the result. The data should consist of the following hexadecimal numbers: 25, 12, 15, 10, 11.

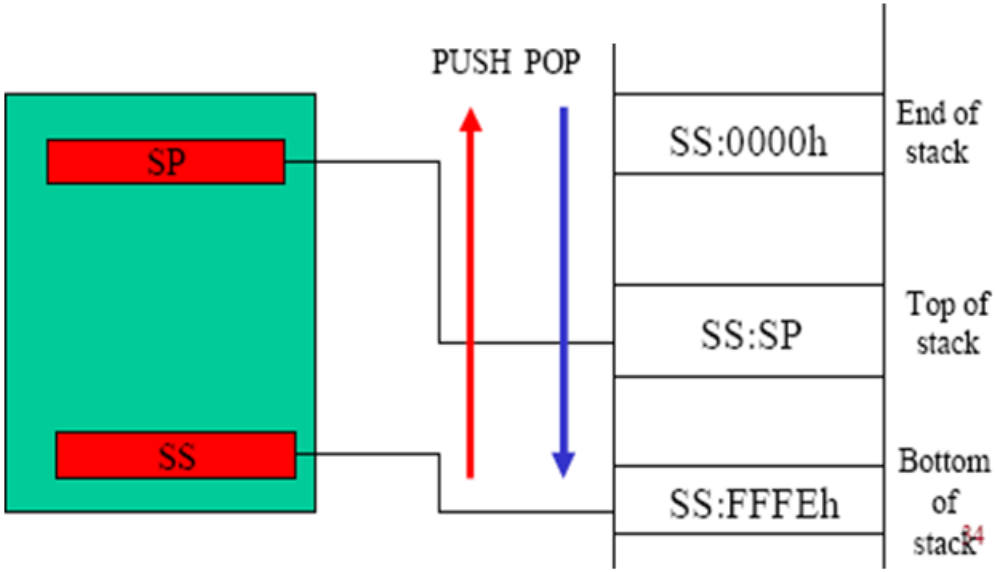
1

```
.model small
.stack 100h
.data
        Data_in DB 25,12,15,10,11
        Sum DB  ?
.code
main proc far
        mov ax, @Data
        mov ds,ax
        mov cx,05h
        mov bx,offset data_in
        mov al,0
```

```
Again:  add al,[bx]
        inc bx
        dec cx
        jnz Again
        mov sum,al
        mov ah,4Ch
        INT 21H
Main    endp
end main
```

The Stack

- The stack is 64 KB used for temporary storage of information such as 32 - bit keywords (KWs)
- Organized from the software point of view as 32 KWs
- SP contains an offset value
- (SS:SP) PA of last storage location in the stack to which data were pushed TOS
- SP is initiated to FFFE (BOS)
- Processor pushes one word at a time
- When a word is pushed, the SP is automatically decremented by 2 and then the word is stored in the stack
- Stack grows from the bottom of the stack SS:FFFE towards the end of stack SS:0000



The Stack (Cont.)

Mnemonic	Meaning	Format	Operation	Flags Affected
PUSH	Push word onto stack	PUSH S	((SP)) ← (S) (SP) ← (SP)-2	None
POP	Pop word off stack	POP D	(D) ← ((SP)) (SP) ← (SP)+2	None

Operand (S or D)
Register Seg-rcg (CS illegal) Memory

Example of PUSH

```
ORG   100h

MOV   AX, 1234h
PUSH  AX          ; store value of AX in stack.

MOV   AX, 5678h   ; modify the AX value.

POP   AX          ; restore the original value of AX.

RET

END
```

The Stack (Cont.)

To save registers and parameters on the stack

PUSH XX
PUSH YY
PUSH ZZ

Push S (16/32 bit or Mem)

(SP) ← (SP)-2
((SP)) ← (S)

Main body of the subroutine

.
.
.
.
.

To restore registers and parameters from the stack and return to the main program

POP ZZ
POP YY
POP XX
RET

Pop D (16/32 bit or Mem)

(D) ← ((SP))
(SP) ← (SP)+2

Subroutines and Subroutine Handling Functions

A subroutine is a specialized segment of a program that can be invoked for execution from any point within the program.

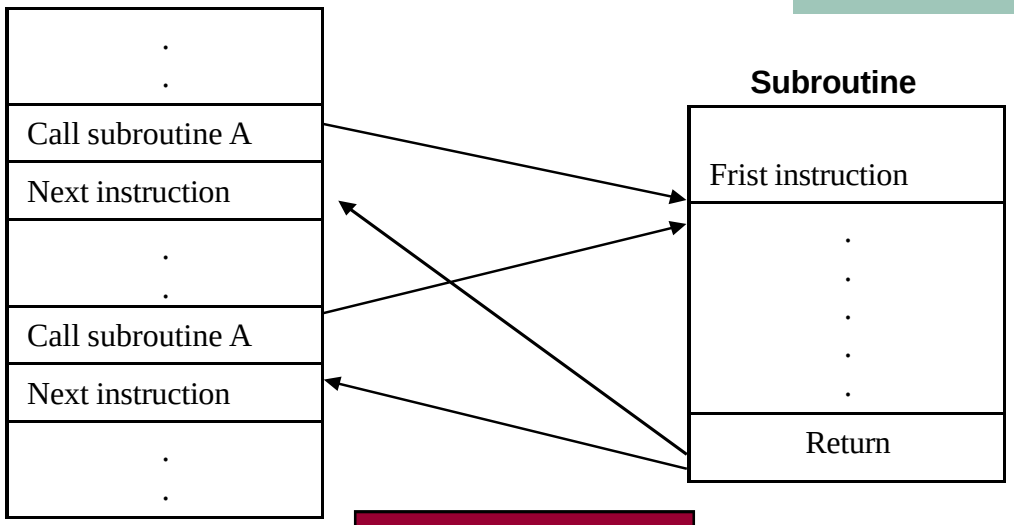
- The **RET** (Return) instruction must be included at the end of the subroutine to facilitate the return to the main program's environment.

Examples of subroutine calls:

- CALL 1234h
- CALL BX
- CALL [BX]

Types of subroutine calls:

1. **Intrasegment Call**: A call within the same code segment.
2. **Intersegment Call**: A call to a subroutine located in a different code segment.



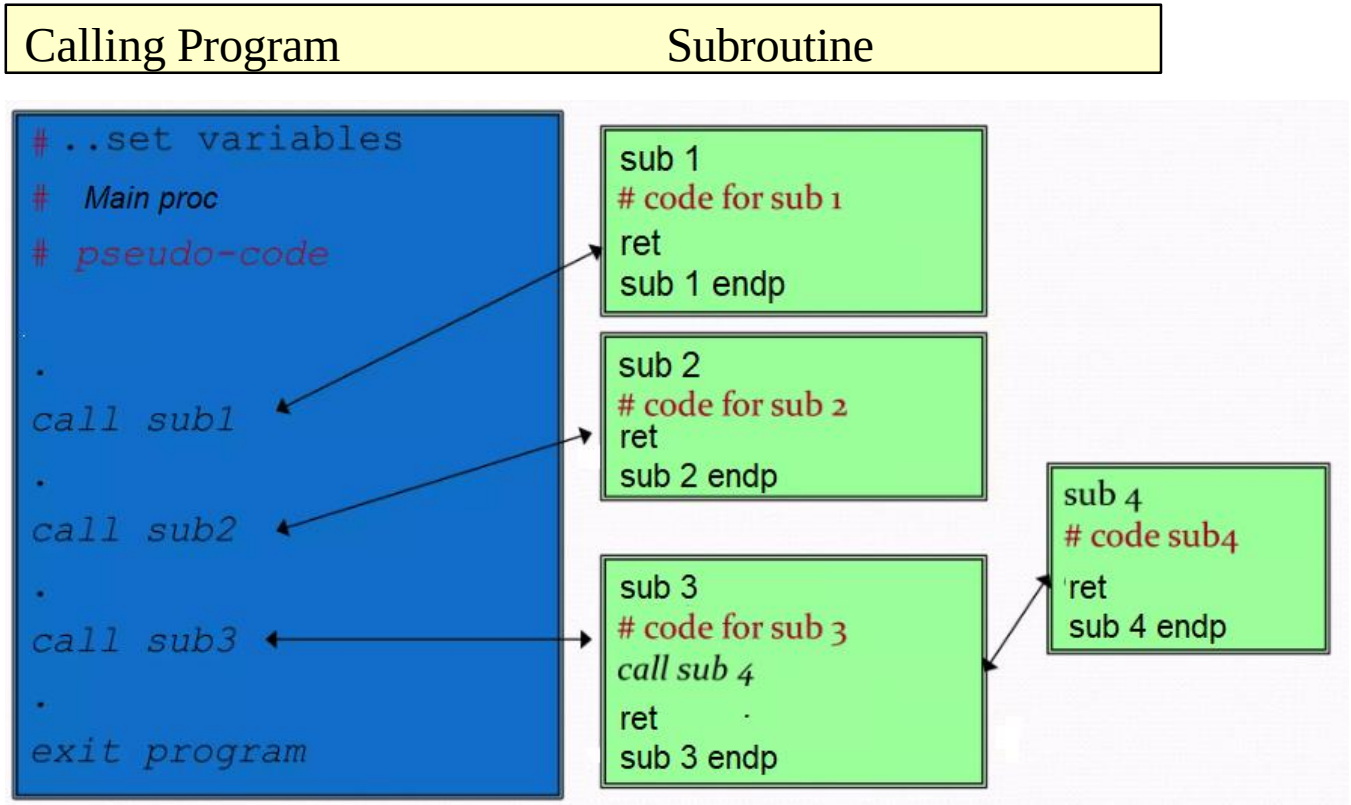
Operand
Near-proc Far-proc Memptr16 Regptr16 Memptr32

Mnemonic	Meaning	Format	Operation	Flags Affected
CALL	Subroutine call	CALL operand	Execution continues from the address of the subroutine specified by the operand. • Critical information, such as the **Instruction Pointer (IP)** and **Code Segment (CS)**, is stored on the stack to enable a return to the main program after the subroutine completes.	None

Calling a NEAR proc

Execution continues from the address of the subroutine specified by the operand.

- Critical information, such as the **Instruction Pointer (IP)** and **Code Segment (CS)**, is stored on the stack to enable a return to the main program after the subroutine completes.



Calling a FAR proc

When calling a **far procedure**, the **CALL** instruction and the subroutine it invokes are located in **different segments**.

- The current values of the **Code Segment (CS)** and **Instruction Pointer (IP)** are saved on the stack.
- The **CS** and the offset of the subroutine are then loaded into the **IP**, enabling execution to jump to the subroutine in the different segment.

Calling Program	Subroutine	Stack

```
Main proc                          sub1  proc
1FCB:001A: call sub1  ---->        4EFA:0080: mov ax,1
1FCB:001D: inc ax  <----           …..
                                   …
 ….                                ret
Main endp                          sub1 endp
```

1ffb	CB
1ffc	1F
1ffd	1D
1ffe	00
1fff	N/A

Nested Procedure Calls

A subroutine can call other subroutines, allowing for **nested procedure**

```
Example:
    Main proc
            000A call subr1
            000C  mov ax,…
              …..
     main endp
```

```
subr2  proc
      0050   nop
            …
      call   subr3
      0060   ret
 subr2 endp
```

Q: show the stack contents at 0079?

1ff0	60
1ffa	00
1ffb	40
1ffc	00
1ffd	00
1ffe	00
1fff	X

```
subr1 proc
      0030   nop
            …
      call subr2
      0040   ret
subr1 endp
```

```
 subr3 proc
        0070   nop
              …
        0079   nop
        007A    ret
subr3  endp
```

Loop and Loop Handling Instructions

Mnemonic	Meaning	Format	Operation
LOOP	Loop	LOOP Short-label	**Purpose:** Decrements the **CX** register and performs a conditional jump. **Behavior:** ○ If **CX** ≠ **0**, execution jumps to the location specified by the short label. ○ If **CX** = **0**, the next sequential instruction is executed.
LOOPE / LOOPZ	Loop while equal / loop while zero	LOOPE/LOOPZ short-label	**Purpose**: Loop while **CX** ≠ **0** and **ZF** = **1** (equal/zero). ○ **Action**: Decrements **CX**. ○ **Behavior**: • If **CX** ≠ **0** and **ZF** = **1**, jump to the short label. • Otherwise, proceed to the next instruction.
LOOPNE / LOOPNZ	Loop while not equal / loop while not zero	LOOPNE/LOOPNZ short-label	**Purpose**: Loop while **CX** ≠ **0** and **ZF=0** (Not Equal/Not Zero). Decrements CX. • **Behavior**: If **CX** ≠ **0** and **ZF** = **0**, jump to the short label. • Otherwise, proceed to the next inst.

Loop

```
NEXT:    MOV CX, COUNT
                .
                .
                .
                .
         LOOP NEXT
```

Load the **count** with the number of repetitions.

Execute the body of the routine that needs to be repeated.

If the **count** is not zero, loop back to the label **NEXT**.

```
         MOV   AX,   DATASEGADDR
         MOV   DS, AX
         MOV   SI,   BLK1ADDR
         MOV   DI,   BLK2ADDR
         MOV   CX, N
NXTPT:   MOV AH, [SI]
         INC SI
         INC DI
         LOOP NXTPT
         HLT
```

Nested Loops

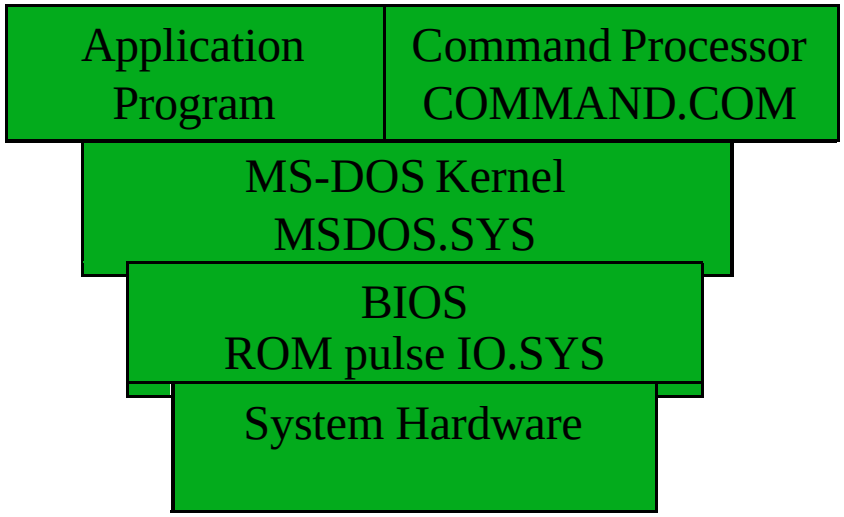
2022 Dr. Hesham EL Zouka

```
MOV CX,A
BACK:…
….
…
…
LOOP BACK
```

Nested Loop

```
        MOV CX,10
OUTER: PUSH CX
        MOV CX,5
INNER: NOP
        …
        …
        …
        LOOP INNER
        POP CX
        LOOP OUTER
```

MS-DOS Functionality and BIOS Interrupt Calls



- BIOS is hardware-specific, tailored to the computer's hardware.
- Supplied by the manufacturer, it is pre-installed on the motherboard.
- The resident portion is stored in ROM for core functions, while the non-resident portion (e.g., IO.SYS) extends BIOS features.

80x86 Interrupts

- An interrupt is an event that causes the processor to suspend its present task and transfer control to a new program called the interrupt service routine (ISR)

- There are three sources of interrupts
  - Processor interrupts
  - Software interrupts
  - Hardware interrupts

- Each interrupt must supply a type number which is used by the processor as a pointer to an interrupt vector table (IVT) to determine the address of that interrupt's service routine

80x86 Interrupts (Cont.)

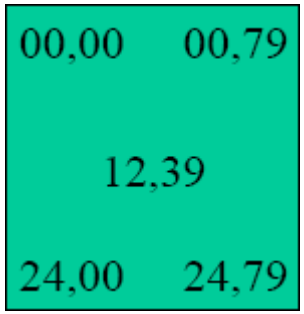
- **An interrupt** is an event that causes the processor to pause its current task and transfer control to a program known as the **Interrupt Service Routine (ISR)**.
- **There are three types of interrupts**:
  - Processor interrupts
  - Software interrupts
  - Hardware interrupts
- Each interrupt provides a **type number**, which the processor uses as a pointer in the **Interrupt Vector Table (IVT)** to locate the address of the corresponding ISR.
- Many vectors in the **Interrupt Vector Table (IVT)** are reserved for specific purposes:
  - **0–1F**: Reserved by the BIOS.
  - **20–3F**: Reserved by the MS-DOS kernel.
- These reserved vectors are used by the processor, BIOS, and kernel for essential system functions.

80x86 Interrupts (Cont.)

- The **BIOS** and **DOS** provide useful subroutines accessible through the **INT (Interrupt)** instruction.
- The **INT** instruction behaves like a **FAR CALL**, saving **CS:IP** and flags on the stack before transferring control to the specified interrupt subroutine.
  - **Format**: **INT xx** (Interrupt number **xx** ranges from 00H to FFH).
  - A total of **256 interrupts** are available.
- Two commonly used interrupts are **INT 10H** (video services) and **INT 21H** (DOS services).
  - To use these services, specific values must be loaded into registers based on the desired function.

INT 10H

- **INT 10H** subroutines are embedded in the **ROM BIOS** of 80x86-based PCs.
- They are used for communication with and control of the computer's screen.



Cursor Locations

INT 10H - Function 06H: Scroll or Clear Screen

This function is used to scroll text in a specified window or clear it entirely.

- **Inputs**:
  - **AL**: Number of lines to scroll.
    - If **AL = 00**, the window will be cleared.
  - **BH**: Attribute for blank lines (e.g., background color).
  - **CH, CL**: Upper-left corner of the window (row and column).
  - **DH, DL**: Lower-right corner of the window (row and column).

INT 10H (Cont.)

- **INT 10H - Function 02H: Set Cursor Position**

This function moves the cursor to a specified screen location.

- **Inputs**:
  - **AH = 02H**: Selects the cursor positioning function.
  - **DH = Row**: Specifies the row number.
  - **DL = Column**: Specifies the column number.

Example:

Write a program to clear the screen and set the cursor at the center:

1. Use **INT 10H Function 06H** to clear the screen.
2. Calculate the center coordinates based on screen size (e.g., **80x25 rows/columns**).
3. Call **INT 10H Function 02H** to place the cursor at the center.

Clearing the Screen and Setting the Cursor at the Center:

```
; Clearing the screen
MOV AX, 0600H; Scroll the entire page
MOV BH, 07; Set normal attribute (W-on-B)
MOV CX, 0000; upper left corner
MOV DX,184FH; lower right corner
INT 10H; Call BIOS interrupt
```

```
; Setting the cursor at the center:
MOV AH,02; Set cursor position function
MOV BH, 00; page 0
MOV DL, 39 Column (39-middle)
MOV DH, 12; Row (12-middle)
INT 10H
```

INT 10H (Cont.)

INT 10H - Function 03H: Get Current Cursor Position

This function retrieves the current position of the cursor on the screen.

- **Inputs**:
  - **AH = 03H**: Selects the function to get cursor position.
  - **BH = 00**: Specifies the page number (usually 0 for standard text mode).
- **Outputs:**
  - **DH**: Contains the row number of the current cursor position.
  - **DL**: Contains the column number of the current cursor position.
  - **CX**: Provides information about the cursor's shape (size, visibility).

```
MOV AH, 03H; Select Get Cursor Position function
MOV BH, 00; Set page number (0)
INT 10H; Call BIOS interrupt to get cursor position
```

```
; After execution:
; DH contains the row, DL contains the column, and CX contains cursor shape info
```

INT 21H

- **INT 21H - DOS Function: Output String to Monitor (Option 09)**
- **INT 21H** is a DOS interrupt that provides various useful system services, such as outputting data to the monitor.

- **Function 09H: Output a String of ASCII Data**
- This function sends an ASCII string to the screen, stopping when the $ character is encountered.

Inputs:

- **AH = 09H**: Specifies the function to output a string.
- **DX**: Contains the offset address of the ASCII string to be displayed.

Code Example:

```
DATA_ASC DB 'The earth is one country','$'; Define the string with $ as the terminator
MOV AH, 09H; Select the function to display a string
MOV DX, OFFSET DATA_ASC; Load the address of the string into DX
INT 21H; Call DOS interrupt to display the string
```

INT 21H (Cont.)

- **INT 21H - DOS Functions for Character I/O**
  - **Option 02H: Output a Single Character to the Monitor**
    This function allows you to display a single character on the screen.

    Inputs:
    - **AH = 02H:** Selects the function to output a single character.
    - **DL:** Contains the ASCII value of the character to be displayed.

  Code Example:

```
MOV AH, 02H; Select output single character function
MOV DL, 'J'; Load the character 'J' into DL
INT 21H; Call DOS interrupt to display the character
```

  - **Option 01H: Input a Single Character with Echo**
    This function waits for a key press, then displays (echo) the pressed character on the screen. After the interrupt, the input character is stored in the AL register.
- **Inputs:**
  - **8AH = 01H:** Selects the function for reading a character with echo.
- **Outputs:**
  - **AL:** Contains the ASCII value of the key pressed.

```
MOV AH, 01H; Select input character with echo
INT 21H; Wait for key press and echo it;
The pressed key will be in AL
```

INT 21H (Cont.)

- **INT 21H - Function 0AH: Input a String from the Keyboard with Echo**

  This function allows input of a string from the keyboard, storing it in a specified buffer. The string input echoes to the screen as the user types.

  Inputs:

  - **AH = 0AH:** Selects the input string function.
  - **DX:** Points to the offset address of the buffer.

  Buffer Structure:

  Byte 1: Maximum size of the buffer (excluding the terminating null).

  Byte 2: Number of characters entered (updated after input).

  Byte 3: The actual characters entered, ending with the carriage return (0DH).

INT 21H (Function 0AH) – Code Example

Code Example:

```
ORG 0010H
DATA1 DB 6 ; Buffer size (max characters to read)
        DB ? ; Placeholder for the number of characters entered
        DB 6 DUP(FFH) ; Space for input data
MOV AH, 0AH ; Select input string with echo function
MOV DX, OFFSET DATA1 ; Load address of the buffer
INT 21H ; Call DOS interrupt to read the string
```

Example Explanation:

If the user types USA followed by <RETURN>, the buffer will be populated as follows:

Address	Value	Explanation
0010H	06	Maximum buffer size
0011H	03	Characters entered
0012H	55H	ASCII for 'U'
0013H	53H	ASCII for 'S'
0014H	41H	ASCII for 'A'
0015H	0DH	Carriage return ('\r')
0016H	FFH	Unused (initial data)